

ReproBench: Benchmarking LLM Agents on From-Scratch IoT Firmware Vulnerability Reproduction

Anonymous submission

Abstract

Large language model (LLM) agents are increasingly evaluated on cybersecurity tasks, yet most benchmarks begin after the target environment has already been prepared: the agent receives source code, a patch, a running service, a container, or an executable binary. This setting leaves open a central question for real-world vulnerability analysis: can an agent reconstruct the missing environment needed to reproduce a known vulnerability from scratch? We introduce REPROBENCH, a public-evidence-grounded benchmark for from-scratch reproduction attempts of IoT firmware vulnerabilities from only a CVE identifier. REPROBENCH contains 30 manually curated IoT CVEs, balanced across buffer overflows, command injections, and authentication bypasses. Each task is accompanied by a manually inspected public-evidence dossier built from CVE records, NVD entries, vendor advisories, public PoCs, exploit databases, blogs, and firmware references. Agents must traverse a six-stage vulnerability reproduction pipeline: information gathering, firmware acquisition, firmware extraction, binary identification, service rehosting, and vulnerability triggering. We propose an evidence-grounded, non-binary scoring framework that separately evaluates planning and execution and requires observable artifacts for every non-zero score. Our evaluation design consists of 450 reproduction runs (30 CVEs, 5 models, 3 trials) and 450 corresponding scoring passes, followed by post-hoc human audit of scoring outputs, R5/R6 claims, and failure modes. REPROBENCH measures whether agents can progressively perform vulnerability reproduction and produce auditable real-target evidence rather than merely final claims, exposing the gap between textual vulnerability understanding and grounded firmware execution.

Introduction

LLM agents now plan, call tools, write code, and operate in software repositories with increasing autonomy. This progress has produced benchmarks for software engineering and cybersecurity agents, including bug fixing, web exploitation, CTF solving, vulnerability reproduction, patching, and exploit construction (Jimenez et al. 2024; Yang et al. 2026; Wang et al. 2025; Zhu et al. 2025; Lee et al. 2026; Zhang

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

et al. 2026; Wang et al. 2026; Lee and Brumley 2026; Pu et al. 2026). A common assumption underlies many of these tasks: the world has already been prepared for the agent. The agent receives a repository, build script, patch, container, harness, running service, or executable target.

Known-vulnerability reproduction in the wild often begins in a less structured state. A security analyst may start with only a CVE identifier and must discover the affected product, locate the correct artifact, recover a runnable environment, and validate vulnerability behavior against the real target. IoT firmware vulnerabilities make this gap especially sharp. Firmware is often distributed as a proprietary binary blob, compiled for MIPS or ARM, packed in vendor-specific formats, dependent on missing NVRAM state and device files, and reachable only after the vulnerable service is rehosted under emulation. Thus firmware reproduction is not merely exploit writing; it is an end-to-end artifact reconstruction and validation task.

We introduce REPROBENCH, a benchmark that evaluates whether LLM agents can move from a bare CVE identifier toward auditable evidence of IoT firmware vulnerability reproduction. This setting follows the spirit of ProgramBench (Yang et al. 2026), which moves beyond localized coding tasks to evaluate full-program reconstruction. REPROBENCH similarly moves beyond prepared vulnerability environments to evaluate the full artifact-to-execution pipeline. The benchmark stresses capabilities that prepared-environment benchmarks largely hide: open-world information gathering, artifact provenance checking, cross-architecture binary analysis, long-horizon tool orchestration, recovery from tool failures, and grounded validation.

The key design choice in REPROBENCH is that it does not assume a prebuilt executable oracle for each vulnerability. Instead, each task is grounded in a manually inspected *public-evidence dossier*: a reference packet of public vulnerability facts and expected evidence types. Agents receive only the CVE identifier, not the dossier. After execution, their plans, traces, workspaces, scripts, logs, reports, and network or process evidence are scored against the dossier. This design reflects the realistic starting point of from-scratch reproduction while avoiding over-claiming that every IoT CVE has been independently re-executed by the benchmark authors.

REPROBENCH also addresses a methodological problem: plausible-looking reproductions are easy to fabricate. When

rehosting is difficult, agents often produce substitutes such as a Python web server that mimics a router endpoint, a synthetic C program that crashes, or a static report with a copied PoC. Such outputs can look successful under weak evaluation, but they do not exercise the real firmware. REPROBENCH therefore uses non-binary, evidence-grounded scoring. It scores both the reproduction plan and the executed task across six stages, while enforcing real-target gating for service rehosting and vulnerability triggering.

Our experimental design contains two linked components. First, agents attempt 30 CVEs with 5 models and 3 trials, producing 450 full traces and workspaces. Second, a rubric-constrained scoring skill evaluates all 450 runs; post-hoc human analysis audits the scoring outputs, suspicious high scores, all R5/R6 claims, and the final failure-mode taxonomy. The central question is not whether an agent writes a convincing final report, but whether its artifacts support each claimed stage of reproduction.

Our contributions are:

- We formulate CVE-only IoT firmware vulnerability reproduction as a from-scratch, long-horizon agent benchmark.
- We construct a manually curated 30-CVE public-evidence dataset balanced across buffer overflow, command injection, and authentication bypass tasks.
- We propose a non-binary, evidence-grounded scoring framework for planning and execution across six dependent stages, with real-target gating for rehosting and triggering.
- We design a 450-run evaluation and post-hoc human audit pipeline to characterize where agents lose auditable evidence and when they substitute simulations for real firmware execution.

Motivation and Task

Why From-Scratch Reproduction?

Existing cybersecurity benchmarks measure important abilities, but typically after major environmental work has been removed. CyberGym and SEC-bench provide source-level projects or benchmark-generated task environments (Wang et al. 2025; Lee et al. 2026); CVE-Bench focuses on web applications with prepared targets (Zhu et al. 2025); ExploitGym and ExploitBench evaluate exploitation after a target is available (Wang et al. 2026; Lee and Brumley 2026); K-Repro starts from kernel patch information and a buildable kernel setting (Pu et al. 2026). These settings test valuable post-environment capabilities, but they do not answer whether agents can create the reproduction environment themselves.

REPROBENCH asks that question directly. The input is a CVE identifier only. The goal is not a textual answer, nor a synthetic demonstration, but auditable evidence that the agent progressed toward reproducing the vulnerability on the real affected firmware service. This makes the task open-world, long-horizon, and artifact-grounded.

Why IoT Firmware?

IoT firmware vulnerabilities provide a compact stress test for many agent capabilities. The agent must retrieve and reconcile public information, verify product and version details, locate firmware images that may have moved or disappeared, distinguish real firmware from HTML error pages, unpack containers and filesystems, reason about non-x86 binaries, and use rehosting tools such as QEMU or firmware emulators. Each stage leaves artifacts that can be audited. This makes the domain suitable for fine-grained evaluation while remaining realistic for defensive tasks such as patch validation and regression testing.

Six-Stage Pipeline

REPROBENCH defines six dependent stages:

1. **R1 Information Gathering:** identify vendor, product, firmware version, vulnerability type, endpoint, and sources.
2. **R2 Firmware Acquisition:** obtain the correct vulnerable firmware image.
3. **R3 Firmware Extraction:** extract a usable filesystem or correctly diagnose encryption.
4. **R4 Binary Identification:** identify architecture, vulnerable service binary, code path, and runtime dependencies.
5. **R5 Service Rehosting:** launch the real firmware service and demonstrate reachability.
6. **R6 Vulnerability Triggering:** execute a PoC against the real rehosted service and observe vulnerability-specific evidence.

The dependency order is strict: R6 is valid only if the real service from R5 is reachable, and R5 is meaningful only after the correct firmware and binary have been identified. This dependency structure is essential for rejecting simulation substitution.

Benchmark Construction

Manual CVE Curation

We construct REPROBENCH through manual curation. Starting from public CVE databases, vendor advisories, NVD entries, exploit databases, security blogs, and GitHub PoC repositories, the authors screen IoT-related vulnerabilities affecting routers, cameras, NAS devices, smart-home hubs, and other embedded firmware devices. A candidate is retained only if it satisfies four criteria: it affects an IoT or embedded firmware product; public references identify the affected product or firmware family; the vulnerability class can be assigned to one of our target categories; and there is enough public evidence to build a reference dossier for scoring.

From the candidate pool, we manually select 30 representative CVEs. The final dataset is balanced by vulnerability class: 10 buffer overflows, 10 command injections, and 10 authentication bypasses. We choose these classes because they are common in IoT firmware and require different validation signals: crashes or abnormal process exits for buffer overflows, command side effects for command injection, and

unauthenticated success on protected operations for authentication bypass. The balanced design does not claim to mirror the natural IoT vulnerability distribution; instead, it creates a controlled benchmark for comparing agent capabilities across three reproduction modes.

Public-Evidence Dossiers

For each CVE, we construct a public-evidence dossier containing vendor and product information, affected firmware versions when available, vulnerability class, affected endpoint or parameter, public references, PoC descriptions when available, firmware acquisition hints, target binary names when known, architecture when available, and expected trigger evidence type. The dossier is used only for scoring; agents receive only the CVE identifier.

The dossier is not a claim that the authors have independently reproduced every CVE end to end. Rather, it defines the publicly documented facts and evidence requirements against which an agent’s artifacts are audited. This distinction is important for IoT: firmware may disappear, vendor links may fail, and public PoCs may omit device state. REPROBENCH measures whether an agent can transform public evidence into auditable artifacts and real-target execution evidence, and it attributes failures to both agent behavior and external blockers such as unavailable firmware or insufficient public detail.

Evidence Scope and Completeness

The public-evidence dossier is organized around the six stages. R1 fields describe facts that can be checked from public records, including vendor, device family, firmware version, endpoint, parameter, and vulnerability class. R2–R4 fields describe artifact expectations, including firmware source hints, expected image names, archive mirrors when known, filesystem format, target binary names, and architecture evidence. R5–R6 fields do not provide an exploit oracle; instead, they specify what evidence would be acceptable if an agent reaches real execution, such as a reachable service banner, handler log, crash signal, command side effect, unauthorized configuration response, or reusable request trace.

This staged dossier design lets us separate three cases that would otherwise be conflated. First, an agent can fail despite strong public evidence, which indicates an agent capability gap. Second, an agent can fail because public artifacts are stale or incomplete, which is recorded as evidence insufficiency or artifact unavailability rather than model-only failure. Third, an agent can generate a plausible report without grounding it in any dossier-supported artifact, which is recorded as simulation substitution or unsupported claim. The benchmark therefore evaluates both progress and epistemic grounding: the score rewards not only what the agent claims, but whether the claim is supported by the dossier and by the agent’s own produced artifacts.

Distinguishing agent search failure from public-evidence insufficiency is inherently imperfect at scale. We therefore use a conservative attribution rule. A run is labeled *artifact unavailable* only when the dossier or post-hoc audit confirms that the referenced source is stale, access-controlled, removed, or returns a non-firmware response, and the agent

Table 1: REPROBENCH dataset composition.

Category	Count	Share
Buffer overflow	10	33.3%
Command injection	10	33.3%
Authentication bypass	10	33.3%
Total	30	100%

attempted at least one reasonable acquisition path. A run is labeled *agent acquisition failure* when the dossier contains an accessible source or enough public hints for acquisition but the trace shows no adequate search, wrong-model download, or premature abandonment. Borderline cases remain in an *ambiguous evidence* bucket and are excluded from model-only failure counts. This does not make availability judgments perfect, but it prevents the benchmark from automatically blaming the model for every missing artifact.

Evidence-Grounded Scoring

Why Non-Binary Scoring?

Binary success is poorly suited to from-scratch firmware reproduction. Most runs fail before the final trigger, but a run that downloads and extracts the correct firmware has progressed much further than one that only reads an advisory. Conversely, an agent can fabricate a mock service and crash it, producing a convincing final report without touching the real target. REPROBENCH therefore scores progress stage by stage and requires observable evidence for every non-zero score.

Plan Score

Before execution, the agent writes a reproduction plan. The plan score has three components. *Coverage* assigns up to 60 points for explicitly covering R1–R6. *Dependency correctness* assigns up to 20 points for respecting the order R1→R2→R3→R4→R5→R6. *Fallback planning* assigns up to 20 points for concrete contingencies, such as archive mirrors when firmware is unavailable, vendor-specific extractors when binwalk fails, or switching between full-system and user-mode emulation when rehosting fails.

Task Score

The task score assigns 100 points across R1–R6: 15 points each for R1–R4 and 20 points each for R5–R6. Each stage is evaluated across five dimensions: correctness (60%), fidelity to the plan (10%), resilience after errors (10%), flexibility in choosing alternatives (10%), and efficiency (10%). Correctness is grounded in stage-specific checklists. For example, R2 checks whether a real firmware file was acquired and whether its version or hash matches reference evidence; R4 checks architecture, candidate binary discovery, vulnerable binary confirmation, and dependency awareness; R5 checks environment setup, process evidence, dependency initialization, service reachability, and proof that the running service is the real firmware service.

Table 2: Task-stage dimensions used for R1–R6.

Dimension	Weight	Meaning
Correctness	60%	Grounded stage outcome
Fidelity	10%	Follows or justifies plan
Resilience	10%	Recovers from errors
Flexibility	10%	Uses alternatives when blocked
Efficiency	10%	Avoids irrelevant work

The overall score is:

$$\text{Overall} = 0.2 \cdot \text{Plan} + 0.8 \cdot \text{Task}. \quad (1)$$

We weight execution more heavily because the benchmark measures whether an agent can realize a reproduction, not merely describe one.

Real-Target Gating

R5 and R6 enforce a real-target requirement. R5 credit requires evidence that the real firmware service was launched and received requests. R6 credit requires that the PoC was executed against that real service and produced vulnerability-specific evidence, such as a crash, command output, file read, or successful unauthenticated operation. Mock servers, synthetic vulnerable programs, host-native simulations, and static-only reports may receive credit for earlier reasoning or PoC construction, but they cannot receive real-target rehosting or triggering credit.

Scoring Skill and Human Audit

We use a rubric-constrained scoring skill to scale evaluation to 450 runs. The scorer receives the public-evidence dossier, plan, trace, workspace artifacts, logs, scripts, and final reports for a single run. It must cite concrete evidence for every non-zero score and assign zero when evidence is missing. The scorer outputs structured JSON and a human-readable summary for each run.

After scoring, human auditors inspect the outputs as a safety layer rather than as a hidden source of success. The audit focuses on all high-scoring runs, all non-zero R5/R6 claims, cases flagged as simulation-like, infrastructure failures, and representative runs from each model and vulnerability class. Auditors check whether the cited evidence supports the assigned stage credit and use the traces to build the final failure taxonomy. Ambiguous claims are adjudicated conservatively: unsupported reproduction claims are not credited.

The audit protocol is intentionally asymmetric: it prevents false positives rather than rescuing missing evidence. If the scorer awards R5, auditors require process, listener, request/response, runtime-log, or equivalent workspace evidence showing that the service belongs to the firmware. If the scorer awards R6, auditors require evidence that the PoC was sent to the real rehosted service and produced behavior consistent with the dossier. Auditors also inspect outliers and map terminal behavior to failure modes, distinguishing capability failures, evidence gaps, and infrastructure failures.

Experimental Design

Agent Runs

We evaluate five model configurations over the 30-CVE benchmark with three independent trials per CVE, for 450 reproduction runs. The evaluated models are `claude-sonnet-4-6`, `deepseek-v4-flash-free`, `glm-5.2`, `gpt-5.5`, and `mimo-v2.5-free`. Each run receives a single CVE identifier, writes a plan, executes in an isolated workspace, and records all artifacts. We preserve the plan, command traces, downloaded files, extracted filesystems, scripts, logs, reports, and any network or process evidence. Infrastructure failures and refusals are retained in the audit trail but interpreted separately from technical reproduction failures.

This design deliberately allows public information use, including advisories and public PoCs. This choice weakens a purist interpretation of “from scratch” if it is read as inventing a PoC without prior examples. We use a different definition: from scratch means the agent receives no prepared target environment, firmware image, harness, or curated task brief beyond the CVE identifier. A public PoC may help with payload syntax, but using it is not itself success. Credit depends on whether the agent grounds that PoC in the correct firmware artifact and, for R6, sends it to a real rehosted service. We therefore report PoC-assisted success separately from non-PoC success. This mirrors real defensive reproduction, where public information is legitimate but does not replace target validation.

Scoring Runs

Every reproduction run is evaluated by the scoring skill, producing 450 JSON score files and 450 summaries. We then aggregate model-level, vulnerability-class-level, and stage-level statistics. The post-hoc human analysis serves three purposes: checking score-evidence consistency, adjudicating real-target claims, and assigning failure modes. Thus the final results combine scalable rubric-based scoring with human analysis of the artifacts most likely to affect conclusions.

Artifact Schema and Audit Trail

Each run is stored as an auditable record rather than as a single final answer. The record contains the initial prompt, the reproduction plan, command and reasoning traces, downloaded files, extraction outputs, scripts written by the agent, process logs, network requests, final reports, scoring JSON, and scoring summaries. This design is important because many invalid reproductions are only visible when the final report is compared with the workspace. For example, a report may claim that a router endpoint crashed, while the workspace shows that the endpoint belonged to a Python mock server created by the agent. Conversely, an agent may fail to produce a polished report but still leave useful evidence such as a correct firmware image, extracted root filesystem, or identified vulnerable binary.

Table 4 summarizes how artifacts are used during scoring and human audit. The scoring skill first assigns stage-level credit using the rubric. Human auditors then inspect the evidence cited by the scorer, prioritizing cases that can change

Table 3: Comparison with representative agent and cybersecurity benchmarks. Prepared environments hide the artifact-to-execution pipeline measured by REPROBENCH.

Benchmark	Input	Prepared env.	Artifact acquisition	Rehosting	Non-binary scoring
SWE-bench (Jimenez et al. 2024)	Repo + issue	Yes	No	No	Tests
ProgramBench (Yang et al. 2026)	Binary + docs	Partial	No	No	Test pass rate
CyberGym (Wang et al. 2025)	Source/harness	Yes	No	No	Staged
CVE-Bench (Zhu et al. 2025)	Web target	Yes	No	No	Binary
ExploitBench (Lee and Brumley 2026)	Built target	Yes	No	No	Capability ladder
K-Repro (Pu et al. 2026)	Patch context	Partial	Limited	Kernel	Mostly final trigger
REPROBENCH	CVE ID	No	Yes	Yes	Stage score

Table 4: Artifacts preserved for scoring and audit.

Artifact	Main use
Plan	Plan coverage, ordering, fallback
Trace	Executed commands and retries
Workspace	Firmware, rootfs, scripts, logs
Reports	Agent claims and final rationale
Network/process logs	R5 service evidence
Crash/side effects	R6 trigger evidence
Scoring JSON	Stage scores and citations

the main conclusions: non-zero R5/R6 scores, high overall scores, simulation-like workspaces, infrastructure failures, and representative failures from each vulnerability class. This audit does not add hidden credit for unobserved work; it only checks whether the evidence already present in the run supports the assigned score.

We ask six research questions:

- RQ1: How far can agents progress from a CVE ID toward IoT firmware vulnerability reproduction?
- RQ2: Which stages account for most failures?
- RQ3: Does plan quality predict execution progress?
- RQ4: How often do agents produce plausible but invalid reproduction artifacts?
- RQ5: How do vulnerability classes and models affect progress?
- RQ6: Does evidence-grounded scoring reveal distinctions hidden by binary success?

Evaluation Results

Overall Capability

Table 5 shows the planned full-experiment reporting format with placeholder values to be replaced after completing all 450 scoring passes. The primary result table reports, for each model, the number of completed runs, average plan score, average task score, average overall score, median overall score, strict real-service success count, strict real-trigger success count, PoC-assisted trigger count, and simulation-substitution rate. We report both stage scores and strict binary counts. Stage scores capture partial progress, while strict counts require direct evidence of a reachable real firmware service and vulnerability-specific behavior.

The expected interpretation of this table is not a leaderboard alone. If the final results follow the pilot pattern, stronger models will improve plan quality and early-stage progress but still show low strict R5/R6 counts. The gap between overall score and strict trigger count is central: it indicates that agents can produce partial evidence without completing real-target vulnerability reproduction.

Stage-Level Funnel

The central analysis is a stage funnel from R1 to R6. Table 6 gives placeholder values for the full-run stage table. We expect early information gathering to be substantially stronger than artifact handling and rehosting, but the 450-run design lets us quantify where the curve drops for each model and vulnerability class. A model that reaches R4 but fails at R5 is different from a model that never acquires firmware; binary success would collapse both to failure.

Plan-Execution Gap

We compare plan score against task score for every run. This analysis separates agents that do not know the reproduction workflow from agents that know the workflow but cannot execute it. The fallback component is especially important: firmware reproduction is dominated by stale links, extraction errors, missing loaders, architecture mismatches, NVRAM initialization failures, and service startup errors. Weak fallback planning predicts premature abandonment and simulation substitution.

Vulnerability-Class Analysis

The balanced dataset enables class-level comparison. Buffer overflows typically require binary confirmation and crash evidence; command injections require service reachability and command side effects; authentication bypasses require stateful request comparisons and protected operation evidence. In the final tables, each class will contain 150 runs; placeholder aggregate overall scores are 40.2 for BOF, 42.6 for CMDI, and 33.5 for authentication bypass, with strict R6 counts of 2, 3, and 1 respectively. For every strict R6 success, we also mark whether the payload was copied or adapted from a public PoC, synthesized by the agent, or derived from static analysis during the run. This separates “environment reconstruction plus script execution” from cases where the agent contributes more of the triggering logic.

Table 5: PLACEHOLDER full-experiment model summary for 450 runs. Values are provisional layout placeholders and should be replaced with completed scoring results.

Model	Runs	Plan	Task	Overall	R5 real	R6 real	Sim. sub.
claude-sonnet-4-6	90	78.4	46.8	53.1	8	3	21.1%
deepseek-v4-flash-free	90	62.7	34.5	40.1	4	1	36.7%
glm-5.2	90	41.3	19.8	24.1	1	0	44.4%
gpt-5.5	90	55.9	27.4	33.1	2	0	40.0%
mimo-v2.5-free	90	66.5	38.2	43.9	5	2	31.1%
All	450	60.9	33.3	38.8	20	6	34.7%

Table 6: PLACEHOLDER stage-level averages over 450 runs.

Stage	Description	Avg. score
R1	Information gathering	12.4 / 15
R2	Firmware acquisition	6.1 / 15
R3	Firmware extraction	5.4 / 15
R4	Binary identification	7.8 / 15
R5	Service rehosting	3.9 / 20
R6	Vulnerability triggering	2.6 / 20

Failure Attribution

The final analysis assigns each failed or partial run a terminal failure mode. Categories include incorrect or missing CVE facts, firmware acquisition failure, extraction failure, binary mismatch, rehosting failure, no real trigger evidence, simulation substitution, public evidence insufficiency, safety refusal, infrastructure failure, and ambiguous acquisition. This attribution is important because not every failed run should be interpreted as the same model weakness: some failures reflect agent decisions, while others reflect missing public artifacts or environmental blockers. Ambiguous acquisition is used when post-hoc audit cannot confidently determine whether the agent failed to search well enough or whether the public link ecosystem was genuinely stale.

Reporting Metrics

We report both graded and strict metrics. Graded metrics include average plan score, task score, overall score, per-stage score, and per-dimension scores for correctness, fidelity, resilience, flexibility, and efficiency. Strict metrics count only runs with direct real-target evidence, such as a reachable firmware service for R5 or a vulnerability-specific trigger against that service for R6. We also report simulation-substitution rate, public-evidence-insufficiency rate, ambiguous-acquisition rate, and PoC-assisted strict-trigger rate. This separation prevents three common misreadings: treating a partial R6 score as a successful exploit, treating every failed run as a model-only failure, and treating public-PoC execution as equivalent to agent-derived vulnerability reproduction.

The main tables therefore answer different questions. Model-level tables ask which agent produces more auditable progress. Stage-level funnels ask where the pipeline col-

lapses. Class-level tables ask whether BOF, CMDI, and authentication bypass tasks stress different skills. Failure-attribution tables ask whether the terminal blocker is an agent decision, an artifact problem, a rehosting problem, a safety refusal, or insufficient public evidence. Together these metrics turn 450 mostly unsuccessful end-to-end attempts into a capability profile rather than a flat zero-success benchmark.

Failure Modes

Our pilot scoring and trace inspection identify four failure families that the 450-run evaluation is designed to quantify.

Artifact failures. Agents download incorrect firmware, stop after vendor 403 errors, or fail to distinguish firmware images from HTML responses. Some runs identify correct CVE facts but never acquire a usable artifact. Public information may also be incomplete, so the audit separates agent failure from evidence insufficiency.

Extraction and binary-analysis failures. Agents misuse extraction tools, stop after incomplete binwalk output, or analyze binaries from PoC repositories rather than from the target firmware. This can produce useful static insight but weakens alignment with the public-evidence dossier and the real target.

The rehosting gap. Agents know the names of QEMU and firmware emulators, but do not reliably configure dynamic loaders, library paths, NVRAM values, startup scripts, device nodes, or network listeners. They often abandon after one or two emulator errors and declare that physical hardware is required.

Simulation substitution. When real execution is difficult, agents generate a simplified substitute that demonstrates the vulnerability pattern but not the vulnerability instance. This is the most important evaluation hazard: the output may be coherent, reproducible, and wrong for the benchmark goal.

Discussion

What REPROBENCH Measures

REPROBENCH does not claim that every CVE has a prebuilt, author-executed oracle. Instead, it evaluates whether agents can produce auditable evidence aligned with public vulnerability facts and real firmware artifacts. This is a deliberate design choice. From-scratch reproduction often begins from incomplete public information, and constructing a fully executable oracle for every IoT firmware CVE would itself require substantial expert effort. REPROBENCH measures the

agent’s ability to transform public evidence into concrete artifacts and real-target evidence under a uniform protocol.

Implications for Agent Evaluation

Prepared environments can hide a major class of agent failures. An agent that succeeds when given a running service may still fail when asked to recover the service from a public vulnerability identifier. Future benchmarks for long-horizon agents should record traces and workspaces, require artifact provenance, distinguish public-evidence insufficiency from agent error, and reject substitutions that do not preserve task validity.

Implications for Agent Design

The benchmark targets recovery-oriented tool use, not only vulnerability knowledge. Firmware reproduction requires error-to-fix mappings, persistent artifact tracking, architecture-aware execution, and self-checks that distinguish real targets from simulations. Rehosting scaffolds may improve absolute performance, but the benchmark should continue to measure whether agents preserve the real-target requirement.

Limitations and Ethics

The dataset is manually balanced across three vulnerability classes and is not intended to match the natural distribution of IoT CVEs. Public-evidence dossiers may contain incomplete or stale information; we mitigate this by requiring explicit evidence for scoring, preserving failure attribution, and auditing scoring outputs after the experiments. The scoring skill is not a deterministic exploit oracle, so human analysis is used to check evidence consistency and adjudicate ambiguous R5/R6 claims. Ethically, REPROBENCH targets defensive reproduction, patch validation, and agent evaluation. Experiments are conducted in isolated workspaces, and real-target credit requires controlled evidence rather than weaponized exploitation.

Related Work

Software and agent benchmarks such as SWE-bench and ProgramBench evaluate code repair or program reconstruction (Jimenez et al. 2024; Yang et al. 2026). REPROBENCH shares ProgramBench’s from-scratch philosophy but focuses on reconstructing vulnerability reproduction environments rather than rebuilding program behavior.

Cybersecurity benchmarks evaluate web exploitation, CTF solving, vulnerability reproduction, patching, and exploit construction (Wang et al. 2025; Zhu et al. 2025; Lee et al. 2026; Zhang et al. 2026; Wang et al. 2026; Lee and Brumley 2026; Pu et al. 2026). These works provide important measurements of agent security capability, but most provide source code, patch context, a harness, an executable target, or a prepared service. REPROBENCH complements them by evaluating the artifact-to-execution pipeline that precedes exploitation in IoT firmware reproduction.

Firmware rehosting tools such as Firmadyne, FirmAE, Greenhouse, and FIRMWELL automate parts of firmware emulation and dependency reconstruction (Chen et al. 2016;

Kim et al. 2020; Tay et al. 2023; Qin et al. 2026). REPROBENCH does not replace these tools; it evaluates whether LLM agents can discover, install, configure, and recover from failures while using such tools in a from-scratch workflow.

Conclusion

REPROBENCH evaluates whether LLM agents can move from a CVE identifier to auditable evidence of IoT firmware vulnerability reproduction. By combining a manually curated 30-CVE public-evidence dataset, 450 reproduction runs, 450 scoring passes, and post-hoc human analysis, it measures progress that binary metrics would obscure. The benchmark is designed to reveal where agents lose grounding: public vulnerability understanding, artifact acquisition, firmware extraction, binary identification, rehosting, or real-target triggering. Its central lesson is that future cyber agents must be evaluated not only on plausible reports or PoC text, but on whether their actions remain grounded in the real artifacts they claim to reproduce.

References

- Chen, D. D.; Woo, M.; Brumley, D.; and Egele, M. 2016. Towards automated dynamic analysis for linux-based embedded firmware. In *NDSS*, volume 1, 1–1.
- Jimenez, C. E.; Yang, J.; Wettig, A.; Yao, S.; Pei, K.; Press, O.; and Narasimhan, K. 2024. Swe-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations*, volume 2024, 54107–54157.
- Kim, M.; Kim, D.; Kim, E.; Kim, S.; Jang, Y.; and Kim, Y. 2020. FirmAE: Towards large-scale emulation of iot firmware for dynamic analysis. In *Proceedings of the 36th Annual Computer Security Applications Conference*, 733–745.
- Lee, H.; Zhang, Z.; Lu, H.; and Zhang, L. 2026. Sec-bench: Automated benchmarking of llm agents on real-world software security tasks. *Advances in Neural Information Processing Systems*, 38: 116342–116378.
- Lee, S.; and Brumley, D. 2026. ExploitBench: A Capability Ladder Benchmark for LLM Cybersecurity Agents. *arXiv preprint arXiv:2605.14153*.
- Pu, J.; Li, X.; Liang, Z.; Cox, J.; Wu, Y.; Shehada, K.; Srivastav, A.; and Qian, Z. 2026. Patch-to-PoC: A Systematic Study of Agentic LLM Systems for Linux Kernel N-Day Reproduction. *arXiv preprint arXiv:2602.07287*.
- Qin, C.; Zhang, C.; Zheng, Y.; Liu, P.; Zhang, J.; Li, Y.; Zhang, W.; Liu, Y.; and Sun, L. 2026. User-Space Dependency-Aware Rehosting for Linux-Based Firmware Binaries. In *NDSS*.
- Tay, H. J.; Zeng, K.; Vadayath, J. M.; Raj, A. S.; Dutcher, A.; Reddy, T.; Gibbs, W.; Basque, Z. L.; Dong, F.; Smith, Z.; et al. 2023. Greenhouse: {single-service} rehosting of {linux-based} firmware binaries in {user-space} emulation. In *32nd USENIX Security Symposium (USENIX Security 23)*, 5791–5808.
- Wang, Z.; Schiller, N.; Li, H.; Narayana, S. S.; Nasr, M.; Carlini, N.; Qi, X.; Wallace, E.; Bursztein, E.; Invernizzi, L.; et al.

2026. ExploitGym: Can AI Agents Turn Security Vulnerabilities into Real Attacks? *arXiv preprint arXiv:2605.11086*.

Wang, Z.; Shi, T.; He, J.; Cai, M.; Zhang, J.; and Song, D. 2025. CyberGym: Evaluating AI Agents' Real-World Cybersecurity Capabilities at Scale. *arXiv preprint arXiv:2506.02548*.

Yang, J.; Lieret, K.; Ma, J.; Thakkar, P.; Pedchenko, D.; Sootla, S.; McMilin, E.; Yin, P.; Hou, R.; Synnaeve, G.; et al. 2026. ProgramBench: Can Language Models Rebuild Programs From Scratch? *arXiv preprint arXiv:2605.03546*.

Zhang, A.; Ji, J.; Menders, C.; Dulepet, R.; Qin, T.; Wang, R.; Wu, J.; Liao, K.; Li, J.; Hu, J.; et al. 2026. Bounty-bench: Dollar impact of ai agent attackers and defenders on real-world cybersecurity systems. *Advances in Neural Information Processing Systems*, 38.

Zhu, Y.; Kellermann, A.; Bowman, D.; Li, P.; Gupta, A.; Danda, A.; Fang, R.; Jensen, C.; Ihli, E.; Benn, J.; et al. 2025. CVE-bench: a benchmark for AI agents' ability to exploit real-world web application vulnerabilities. *arXiv preprint arXiv:2503.17332*.